AML 2503 – SQL & NoSQL Database Design

Assignment 02 – W2026

Group 2 (4 members)

FoodFleet NoSQL Database Design

1. Justification for Choosing a NoSQL Database

FoodFleet is a modern food delivery platform that must manage highly varied and fast-changing data. The system includes user profiles, restaurant profiles, restaurant menus, customized meals, orders from multiple restaurants, delivery drivers, order logs, and real-time notifications. This data is heterogeneous because each restaurant can structure its menu differently, users can store different preferences and dietary filters, and each order may contain different nested items, options, and delivery events. The assignment itself emphasizes this heterogeneity, semi-structured data, real-time delivery tracking, mixed-cart ordering, and fast-changing activity logs.

A relational database can manage structured data very well, but FoodFleet's requirements create several problems for a purely relational model. First, restaurant menus are not uniform. One restaurant may offer simple fixed meals, while another may offer deeply customizable dishes with sizes, toppings, spice levels, add-ons, and dietary tags. In a relational model, this would often require many related tables and frequent schema changes. Second, order logs and delivery status updates grow rapidly and continuously, especially during peak traffic. Third, recommendation-related attributes and user preferences may evolve over time, making rigid schemas harder to maintain. MongoDB's document model is designed for flexible, evolving structures and supports embedding arrays and subdocuments so related data can be retrieved together with fewer joins.

NoSQL databases were introduced in part to address flexibility and scalability limitations that appear in highly distributed and fast-changing workloads. Systematic reviews and surveys describe NoSQL systems as better suited than traditional rigid schemas when applications need horizontal scalability, evolving data models, and high-volume operational workloads.

For FoodFleet, scalability is essential because the platform may face heavy concurrent traffic during lunch, dinner, weekends, and promotions. The database must also support frequent menu

updates, rapid order creation, live delivery tracking, and notification delivery. MongoDB Atlas is appropriate here because it combines a document-oriented model with cloud deployment, indexing, replication, and scaling features useful for real-time application workloads. MongoDB's documentation also recommends shaping documents around application access patterns and using embedding when related data is often read together.

Therefore, a NoSQL solution is justified because it provides:

- schema flexibility for diverse menus and customizable meals,
- efficient storage of nested order/cart structures,
- support for rapidly growing logs and notifications,
- easier adaptation when new app features are introduced,
- better alignment with real-time, high-volume application behavior.

2. Selected NoSQL Type

Chosen Type: Document-Oriented NoSQL Database

Recommended Platform: MongoDB Atlas

A **document-oriented database** is the best choice for FoodFleet because most core business entities naturally fit JSON-like documents. Users, restaurants, menus, orders, drivers, and notifications all contain nested and semi-structured information. MongoDB stores data as BSON documents, which supports arrays, subdocuments, and flexible fields. This is especially useful for representing:

- restaurant menus with variable structures,
- customizable meal options,
- mixed-cart orders from multiple restaurants,
- delivery logs with timestamps and GPS locations,
- notification content with different message types.

Why document-oriented is best for FoodFleet

1. Natural fit for nested business data
Orders contain cart items, customizations, payment info, split-bill details, and delivery events. In MongoDB, this can be stored in one logical document instead of spreading it across many tables.

2. Flexible menu design
Restaurant A may sell burgers with extra toppings, Restaurant B may sell pizza with crust choices, and Restaurant C may sell meal bowls with protein/base/sauce combinations. A document model handles all of these without redesigning the whole schema.

3. Good performance for app-style reads
Food delivery apps commonly retrieve one "whole object" at a time, such as a user profile, a

restaurant page, or an order with its items and status. Embedding helps serve such reads efficiently.

Comparison with another NoSQL paradigm: Graph database

A **graph database** such as Neo4j would be strong for advanced recommendation engines because it models relationships among users, foods, cuisines, orders, and restaurants very well. It may outperform a document database for deep relationship traversal such as “users who liked dishes similar to this one also ordered from these restaurants.” However, graph databases are less natural for storing the main operational transaction documents such as menus, carts, order payloads, and delivery logs. For FoodFleet’s core operational system, document storage is a better primary choice, while graph could be an optional future secondary system for advanced recommendations.

Trade-off summary

- **Document DB strengths:** flexibility, nested data, operational simplicity, fast application reads.
- **Graph DB strengths:** advanced relationship analytics and recommendation traversal.
- **Best choice for this case: MongoDB Atlas** as the primary NoSQL platform.

3. NoSQL Logical Schema Design

Below is a concise JSON-like logical schema.

3.1 Users Collection

```
{
  "_id": ObjectId("..."),
  "user_id": "U001",
  "full_name": "Customer A",
  "email": "customer.a@email.com",
  "phone": "+1-613-000-0000",
  "addresses": [
    {
      "label": "Home",
      "street": "123 Main St",
      "city": "Ottawa",
      "province": "ON",
      "postal_code": "K1A1A1",
      "location": { "type": "Point", "coordinates": [-75.6972, 45.4215] }
    }
  ],
  "dietary_preferences": ["halal", "gluten-free"],
  "favourite_foods": ["shawarma", "salad", "pizza"],
  "payment_methods": [
    {
      "method_id": "PM001",
```

```
      "type": "credit_card",
      "last4": "4242",
      "card_holder": "Customer A"
    }
  ],
  "order_history": ["O001", "O002", "O005"],
  "created_at": ISODate("2026-03-01T10:00:00Z")
}
```

Embedding used: addresses, payment methods
Referencing used: order history references order IDs

3.2 Restaurants Collection

```
{
  "_id": ObjectId("..."),
  "restaurant_id": "R001",
  "name": "Ottawa Green Bowl",
  "cuisine_type": ["Healthy", "Vegetarian"],
  "contact": {
    "phone": "+1-613-111-1111",
    "email": "info@greenbowl.ca"
  },
  "address": {
    "street": "100 Bank St",
    "city": "Ottawa",
    "province": "ON",
    "postal_code": "K1P1A1",
    "location": { "type": "Point", "coordinates": [-75.7000, 45.4200] }
  },
  "open_hours": {
    "mon_fri": "10:00-22:00",
    "sat_sun": "11:00-23:00"
  },
  "rating": 4.6,
  "is_active": true,
  "menus": [
    {
      "menu_id": "M001",
      "category": "Dinner",
      "items": [
        {
          "item_id": "I001",
          "name": "Protein Bowl",
          "description": "Custom bowl with rice, greens, protein and sauce",
          "base_price": 13.99,
          "dietary_tags": ["gluten-free"],
          "customizable_options": {
            "protein": ["chicken", "tofu", "falafel"],
            "base": ["rice", "quinoa", "greens"],
            "sauce": ["tahini", "spicy mayo"]
          },
          "dynamic_pricing": {
            "peak_hour_surcharge": 1.50
          },
          "available": true
```

```
        }
      ]
    }
  ]
}
```

Embedding used: menu categories and items inside restaurant
Reason: menus are usually read together with the restaurant page and can vary widely.
MongoDB recommends embedding when related data is usually read together.

3.3 Orders Collection

```
{
  "_id": ObjectId("..."),
  "order_id": "O003",
  "user_id": "U001",
  "restaurants": ["R001", "R002"],
  "group_order": {
    "is_group_order": true,
    "group_id": "G001",
    "participants": [
      { "user_id": "U001", "share_amount": 18.50 },
      { "user_id": "U002", "share_amount": 21.00 }
    ]
  },
  "cart_contents": [
    {
      "restaurant_id": "R001",
      "items": [
        {
          "item_id": "I001",
          "item_name": "Protein Bowl",
          "quantity": 1,
          "selected_options": {
            "protein": "tofu",
            "base": "quinoa",
            "sauce": "tahini"
          },
          "unit_price": 13.99
        }
      ]
    },
    {
      "restaurant_id": "R002",
      "items": [
        {
          "item_id": "I010",
          "item_name": "Gluten-Free Pasta",
          "quantity": 1,
          "selected_options": {
            "size": "regular"
          },
          "unit_price": 15.50
        }
      ]
    }
```

```
  ],
  "subtotal": 29.49,
  "tax": 3.83,
  "delivery_fee": 4.99,
  "total": 38.31,
  "payment_status": "paid",
  "delivery_status": "out_for_delivery",
  "driver_id": "D001",
  "created_at": ISODate("2026-03-20T18:20:00Z"),
  "estimated_delivery_time": ISODate("2026-03-20T18:55:00Z"),
  "log_ids": ["L1001", "L1002", "L1003"]
}
```

Embedding used: cart contents, split info
Referencing used: log IDs, driver ID, user ID, restaurant IDs

3.4 Delivery Drivers Collection

```
{
  "_id": ObjectId("..."),
  "driver_id": "D001",
  "full_name": "Alex Khan",
  "phone": "+1-613-222-2222",
  "vehicle_type": "car",
  "current_status": "delivering",
  "current_location": {
    "type": "Point",
    "coordinates": [-75.6890, 45.4170]
  },
  "assigned_deliveries": ["O003", "O004"],
  "rating": 4.8,
  "availability": true
}
```

3.5 Order Logs Collection

```
{
  "_id": ObjectId("..."),
  "log_id": "L1002",
  "order_id": "O003",
  "status": "picked_up",
  "timestamp": ISODate("2026-03-20T18:35:00Z"),
  "location": {
    "type": "Point",
    "coordinates": [-75.6950, 45.4195]
  },
  "note": "Driver picked up all items from restaurants"
}
```

Why reference logs instead of embed all logs into orders?
Because logs can grow continuously and become unbounded. MongoDB documentation recommends using references when data grows heavily or when embedding would produce large mutable arrays.

3.6 Notifications Collection

```
{
  "_id": ObjectId("..."),
  "notification_id": "N001",
  "user_id": "U001",
  "type": "order_update",
  "title": "Your order is on the way",
  "message": "Driver Alex has picked up your order.",
  "related_order_id": "O003",
  "is_read": false,
  "created_at": ISODate("2026-03-20T18:36:00Z")
}
```

Embedding vs Referencing Justification

For FoodFleet, a hybrid design is best.

Use Embedding for:

- addresses inside users,
- payment methods inside users,
- menus inside restaurants,
- cart items inside orders,
- group-order participant split details inside orders.

This is appropriate because these pieces are tightly related and are commonly read together in one screen or one API response. Embedding reduces joins and supports single-document reads and atomic updates for closely related data.

Use Referencing for:

- order logs,
- users to orders,
- drivers to assigned deliveries,
- notifications to orders,
- restaurants linked from mixed-cart orders.

This is appropriate when data grows quickly, changes independently, or creates many-to-many style relationships. MongoDB specifically notes that references are useful when embedding would cause excessive duplication or very large growing arrays.

Sample Collections Required by the Assignment

The assignment requires at least:

- **3 users**
- **3 restaurants**
- **5 orders** including **1 group order**
- **2 drivers**
- **5 notifications**

We can use these example IDs:

Users

- U001 – Customer A
- U002 – Customer B
- U003 – Customer C

Restaurants

- R001 – Ottawa Green Bowl
- R002 – Pasta Palace
- R003 – Spice Route Grill

Drivers

- D001 – Alex Khan
- D002 – Emma Roy

Orders

- O001 – single restaurant order
- O002 – single restaurant order
- O003 – mixed-cart group order
- O004 – single restaurant order
- O005 – single restaurant order

Notifications

- N001 to N005 – mix of order updates and promotions

4.Implement the Database in MongoDB Atlas

Ten Key MongoDB Queries:

Here are strong sample queries for my notebook.

1. Retrieve all ongoing deliveries for a user

```
db.orders.find({
  user_id: "U001",
  delivery_status: { $in: ["preparing", "picked_up", "out_for_delivery"] }
})
```

2. Find all gluten-free dinner options across restaurants

```
db.restaurants.find(
  {
    "menus.category": "Dinner",
    "menus.items.dietary_tags": "gluten-free"
  },
  {
    name: 1,
    "menus.$": 1
  }
)
```

3. Show all order status updates for a specific group order

```
db.order_logs.find({ order_id: "O003" }).sort({ timestamp: 1 })
```

4. Find all orders assigned to a specific driver

```
db.orders.find({ driver_id: "D001" })
```

5. Retrieve unread notifications for a user

```
db.notifications.find({
  user_id: "U001",
  is_read: false
})
```

6. Find restaurants open now

```
db.restaurants.find({ is_active: true })
```

7. Get all vegetarian items from a specific restaurant

```
db.restaurants.find(
  { restaurant_id: "R001" },
  { name: 1, "menus.items": 1 }
)
```

8. Count orders per restaurant

```
db.orders.aggregate([
  { $unwind: "$restaurants" },
  { $group: { _id: "$restaurants", total_orders: { $sum: 1 } } }
])
```

9. Find top-rated available drivers

```
db.drivers.find({
  availability: true
}).sort({ rating: -1 })
```

10. Retrieve a user's complete order history

```
db.orders.find({ user_id: "U001" }).sort({ created_at: -1 })
```

CRUD Operations to Demonstrate in MongoDB Atlas

Evidence of operational use:

Create

```
db.users.insertOne({
  user_id: "U004",
  full_name: "New Customer",
  dietary_preferences: ["vegan"]
})
```

Read

```
db.users.find({ user_id: "U001" })
```

Update

```
db.orders.updateOne(
  { order_id: "O003" },
  { $set: { delivery_status: "delivered" } }
)
```

Delete

```
db.notifications.deleteOne({ notification_id: "N005" })
```

A link to a shared Google Colab notebook:
Group2_AML2503_Assignment02.ipynb
(Notebook link removed from this public copy.)

Conclusion

In conclusion, MongoDB Atlas is an appropriate NoSQL solution for FoodFleet because the platform manages semi-structured, nested, and rapidly changing data. A document-oriented model fits the operational needs of users, restaurants, menus, orders, drivers, and notifications better than a rigid relational structure for this use case. The proposed schema uses embedding where data is closely related and frequently read together, while references are used for large and fast-growing entities such as order logs and linked operational records. This hybrid approach improves flexibility, supports real-time delivery workflows, and provides a scalable foundation for future features such as personalization and recommendation services.

Peer-Reviewed Sources/References:

1. Cattell, R. (2011). *Scalable SQL and NoSQL data stores*. ACM SIGMOD Record.
2. Vera-Olivera, H., et al. (2021). *Data Modeling and NoSQL Databases - A Systematic Literature Review*. ACM Computing Surveys.
3. Makris, A., et al. (2021). *MongoDB vs PostgreSQL: A comparative study on performance*. GeoInformatica.
4. Sen, P. S., et al. (2024). *An ontology-based approach to designing a NoSQL database*. Cluster Computing.